
Concurrencia y Paralelismo

Grado en Ingeniería Informática

Julio 2021

1. Clientes en múltiples colas [2.5 puntos]

Queremos implementar un sistema donde una serie de clientes esperan a ser atendidos por un servidor. Los clientes esperan en una cola indicada por `c→queue_number` de entre `N` posibles, numeradas de 0 a `N-1`. Dentro de cada cola los clientes deben ser atendidos en orden de llegada, y una vez son avisados por el servidor llaman a `get_service()`.

El servidor atiende a un cliente de cada vez, rotando entre las colas, es decir, atendiendo un cliente de la cola 0, después de la 1, y así sucesivamente. Si una cola está vacía, el servidor debe pasar a la siguiente. No es necesario controlar el caso de que todas las colas estén vacías. Para simular el proceso de atender al cliente el servidor llama a `serve()`. Las llamadas a `serve()` y `get_service()` deben hacerse sin ningún mutex bloqueado.

```
#define N ...

void insert(queue *q, void *);
void *remove(queue *q);
int elements(queue *q);

struct customer {
    int queue_number;
};

queue *q[N];

void *customer(void *ptr) {
    struct customer *c = ptr;
    ...
    get_service();
}

void *server(void *ptr) {
    while(1) {
        ...
        serve();
    }
}
```

Complete la implementación de `customer` y `server` para que simulen el comportamiento descrito.

2. Threads que simulan corredores de una carrera por relevos [2.5 puntos]

a) Crear la función `runner()` que simula un corredor en una carrera de relevos.

Cosas a tener en cuenta:

- No pueden usarse variables globales.
- No puede hacerse espera activa.
- El primer (1) y el último (N) corredor son especiales.
- El primero tiene que esperar a que se dé la salida, y que ésta sea válida. Si no es válida hay que repetir la carrera. La variable `valid_start` indica si la salida es válida. La variable de condición `start` se lanza cada vez que hay una salida.
- El último corredor tiene que imprimir un mensaje cuando llega a meta.
- Cada corredor imprimirá desde que hora está listo, a que hora recibe el testigo y a que hora termina su relevo. Usar la función `seconds_since_start()` para saber cuantos segundos han pasado desde el comienzo de la carrera.
- Los corredores tienen que correr en orden de número (parámetro `id_runner`).
- Cada corredor tiene que pasar el testigo al siguiente corredor. Pista, úsese un campo en la estructura `token`.
- El programa tiene que funcionar incluso si un corredor se despierta en un momento en que no le toque correr.
- Cada corredor tiene que ejecutar la función `run_100_meters()`, que es la que simula que ha corrido los metros que le corresponde.

```
struct token {
    int total_num_runners;
    pthread_cond_t start;
    bool valid_start;
};

void runner(int id_runner, struct token *t)
{
    ...
    run_100_meters();
    ...
}
int seconds_since_start(void);
```

b) Modificar la implementación anterior para que se pueda usar `signal()` en vez de `broadcast()`, es decir, cada corredor despierta solo al siguiente. Tiene que seguir funcionando en el caso de que a un corredor se le despierte cuando no le toca correr.

```
int pthread_cond_broadcast(pthread_cond_t *cond);
int pthread_cond_signal(pthread_cond_t *cond);
int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex);
int pthread_mutex_lock(pthread_mutex_t *mutex);
int pthread_mutex_unlock(pthread_mutex_t *mutex);
int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
    void *(*fun) (void *), void *arg);
int pthread_join(pthread_t thread, void **retval);
int pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutexattr_t *attr);
int pthread_mutex_destroy(pthread_mutex_t *mutex);
int pthread_cond_init(pthread_cond_t *cond, pthread_condattr_t *cond_attr);
int pthread_cond_destroy(pthread_cond_t *cond);
```

EXAMEN DE CONCURRENCIA Y PARALELISMO

Bloque II: Paralelismo

APELLIDOS: _____ NOMBRE: _____

Convocatoria extraordinaria

9 de julio de 2021

AVISO: No mezclas en el mismo folio preguntas del bloque de concurrencia y del bloque de paralelismo. Las respuestas a cada bloque se entregan por separado.

- [2,5p] 1. **Conceptos de paralelismo.** El temido capitán pirata BarbaFIC, el más inteligente del Atlántico, tiene un método diferente a los demás para esconder los tesoros que roba de otros barcos. En lugar de guardar el tesoro en un único cofre, lo distribuye en muchas bolsitas pequeñas que va enterrando a lo largo y ancho de una isla. Como es muy desconfiado, no guarda las indicaciones de dónde están los fragmentos del tesoro en un mapa sino que, llegado el momento de recuperarlo, realiza los siguientes pasos: 1) Analiza el mapa de la isla, dividiéndola en fragmentos de igual tamaño; 2) Va cavando hoyos en cada uno de estos fragmentos, uno tras otro, para encontrar, en caso de que lo hubiese, parte del tesoro enterrado.

Durante muchos años, todo el proceso de recuperación del tesoro lo hacía el solo, porque tampoco confiaba en su tripulación. Sin embargo, al hacerse mayor, el segundo paso le suponía mucho esfuerzo, así que decidió dividir el trabajo de cavar los fosos (segundo paso) con parte de la tripulación. Eso sí, el análisis del mapa, para el que siempre necesitaba $\frac{1}{10}$ del tiempo total, siguió haciéndolo él en exclusiva.

Contesta DE FORMA RAZONADA las siguientes preguntas:

- a) ¿Cuál es la aceleración máxima que puede obtener BarbaFIC en el proceso completo de búsqueda del tesoro en una determinada isla gracias a dividir la segunda fase entre varios piratas? [0,5p]
- b) A partir de ahora para el segundo paso usa el siguiente proceso:
 - Asigna a cada miembro de la tripulación un fragmento inicial para cavar. BarbaFIC no cava debido a su avanzada edad, solo coordina.
 - Una vez han terminado de cavar el fragmento asignado, todos los marineros deben volver a la playa donde se encuentra BarbaFIC, quien les asignará un nuevo fragmento mientras quede alguno de la isla por explorar.¿Qué tipo de descomposición y asignación de tareas está usando BarbaFIC? [0,5p]
- c) BarbaFIC y su tripulación llegan a una isla que se divide en 100 fragmentos iguales, y donde el tiempo de explorar cada uno de estos fragmentos es homogéneo (90 minutos por fragmento). Además, entre exploración y exploración de fragmento BarbaFIC los marineros necesitan 10 minutos para ir a la playa, recibir instrucciones y estar listos para la nueva excavación. ¿Cuál es la aceleración y la eficiencia de la aproximación del apartado anterior si trabajan cuatro marineros además de BarbaFIC (este último solo en tareas de coordinación). [0,5p]
- d) ¿Cuál es el mínimo número de marineros que necesitaría BarbaFIC para hacer la búsqueda del tesoro en esta isla al menos cuatro veces más rápido que si todo el proceso lo hiciera solo BarbaFIC? [0,5p]

- e) Indica, de palabra, una optimización que se podría realizar al proceso de búsqueda paralelo del apartado b) [0,5p]

[2,5p] 2. **Diseño de algoritmos paralelos.** Queremos paralelizar el siguiente código en un sistema paralelo de P procesadores donde el procesador 0 es el único que tiene acceso al disco duro y N es conocido de antemano y podemos asumir que es múltiplo de P:

```
void main(int argc, char *argv[]) {
    double max_desv, media = 0., max = 0., min = 0.;
    double u[N][N];

    LeeMatriz(u);

    // calcular estadísticas
    for(int i=0; i<N; i++){
        for(int j=0; j<N; j++){
            media = media + u[i][j];
            if(u[i][j] > max) max = u[i][j];
            if(u[i][j] < min) min = u[i][j];
        }
    }
    media = media / (N*N);
    max_desv = ((max - media) > (media - min)) ? (max - media) : (media - min);

    // modificar matriz
    for(int i=0; i<N; i++){
        for(int j=0; j<N; j++){
            u[i][j] = u[i][j] - max_desv;
        }
    }

    EscribeMatriz(u);
}
```

- a) Descomposición y asignación de tareas justificadas. [0,5p]
b) Código MPI reflejando la descomposición y asignación propuesta, sin colectivas [1,25p]
c) 'Es posible mejorar el código mediante colectivas? De ser así escribe la versión correspondiente en MPI [0,75p]

```
int MPI_Send(void *buf, int count, MPI_Datatype datatype,
             int dest, int tag, MPI_Comm comm)
int MPI_Recv(void *buf, int count, MPI_Datatype datatype,
             int source, int tag, MPI_Comm comm, MPI_Status *status)
int MPI_Bcast(void *buffer, int count, MPI_Datatype datatype,
             int root, MPI_Comm comm)
int MPI_Scatter(void *sendbuf, int sendcnt, MPI_Datatype sendtype,
               void *recvbuf, int recvcnt, MPI_Datatype recvtype,
               int root, MPI_Comm comm)
int MPI_Gather(void *sendbuf, int sendcnt, MPI_Datatype sendtype,
               void *recvbuf, int recvcnt, MPI_Datatype recvtype,
               int root, MPI_Comm comm)
int MPI_Allreduce(void *sendbuf, void *recvbuf, int count,
                  MPI_Datatype datatype, MPI_Op op, MPI_Comm comm)
int MPI_Reduce(void *sendbuf, void *recvbuf, int count,
               MPI_Datatype datatype, MPI_Op op,
               int root, MPI_Comm comm)
```

SOLUCIÓN

1. a) La tarea de análisis del mapa de la isla no es paralelizable, ya que siempre la va a realizar BarbaFIC. Esta tarea consume $\frac{1}{10}$ del tiempo total. Por tanto, de acuerdo a la ley de Amdahl, la máxima aceleración será:

$$\frac{\frac{1}{10} + \frac{9}{10}}{\frac{1}{10}} = 10$$

- b) BarbaFIC aplica una descomposición de dominio (divide la isla en fragmentos de igual tamaño) y una asignación dinámica del tipo Maestro-Esclavo.
- c) Primero empezamos calculando el tiempo secuencial. Sabemos que hay 100 fragmentos que explorar, y cada uno necesita 90 minutos, así que el proceso de cavar la isla completa por una única persona necesitaría de 9000 minutos. Esto es el 90 % del tiempo secuencial, porque el 10 % es el análisis inicial del mapa. Lo que quiere decir que el tiempo secuencial total es 10000 minutos.

Si tenemos cuatro marineros además de BarbaFIC, esto quiere decir que cada uno explorará 25 fragmentos, necesitando un tiempo de $25 * 90 = 2250$ minutos. Además, cada uno debe volver hasta la playa y recibir órdenes en 24 ocasiones, lo que implica un tiempo adicional de $24 * 10 = 240$ minutos. Así el tiempo paralelo de la segunda fase serán 2490 minutos, a los que hay que sumarle los 1000 minutos de la fase inicial que siempre hará BarbaFIC en exclusiva: 3490 minutos.

Por tanto, la aceleración será: $Acc = \frac{10000}{3490} = 2,87$

Y la eficiencia: $Eff = \frac{2,87}{4} = 0,72$

- d) Si tenemos n marineros, cada uno explorará $\frac{100}{n}$ fragmentos, y volverá a recibir órdenes de BarbaFIC $\frac{100}{n} - 1$ veces. Eso quiere decir un tiempo paralelo en la fase de exploración de $\frac{100}{n} * 90 + (\frac{100}{n} - 1) * 10$. Al tiempo paralelo total hay que añadirle los 1000 minutos de analizar el mapa:

$$T_{par}(n) = 1000 + \frac{100}{n} * 90 + (\frac{100}{n} - 1) * 10$$

Así que habría que despejar la ecuación resultante de:

$$\frac{T_{seq}}{T_{par}} = 4$$

$$\frac{10000}{1000 + \frac{100}{n} * 90 + (\frac{100}{n} - 1) * 10} = 4$$

De esa ecuación sale $n = 6,62$ lo que indica que el número mínimo de marineros a utilizar es 7.

- e) Lo lógico es intentar minimizar el tiempo “perdido” por acudir tantas veces a recibir nuevas órdenes de BarbaFIC, para que los piratas puedan cavar distintos fragmentos de una forma más seguida. Una opción es que BarbaFIC, en lugar de indicar un fragmento de cada vez a cada pirata, ya le diga un conjunto de fragmentos.

Otra opción es hacer un reparto de los fragmentos de forma estática, asignando la misma cantidad de fragmentos a cada marinero ya inicialmente, y que no vuelvan a la zona de mando hasta que hayan terminado. Esta opción solo tendría un buen rendimiento si el tiempo de cavar en todos los fragmentos es el mismo (como se indica en el apartado c).

2. a) La descomposición en tareas a alto nivel sigue un modelo híbrido con 4 fases o tareas: lectura de la matriz (T1), cálculo de estadísticas (T2), modificación de la matriz (T3), y escritura de la matriz (T4).

A su vez las tareas T2 y T3 se dividen en subtareas. En el caso de T3 se trata claramente de una descomposici3n de dominio con operaciones paralelas repetitivas sobre una estructura de datos. En el caso de T2 se podr3 argumentar que es una descomposici3n divide-y-vencers ya que el clculo de las estadsticas de media, mximo y mnimo puede considerarse como un problema de este tipo en el que calculamos la media de submatrices, que son partes del problema, para luego unir esos resultados parciales para obtener las estadsticas globales. La asignaci3n es esttica porque todos los elementos requeridos para asignar las tareas son conocidos de antemano.

```

void main(int argc, char *argv[]) {
    int rank, P;
    MPI_Status status;
    double max_desv, media = 0., max = 0., min = 0.;
    double u[N][N], buf[3];

    MPI_Init(&argc, &argv);
    MPI_Comm_size(&P, MPI_COMM_WORLD);
    MPI_Comm_rank(&rank, MPI_COMM_WORLD);

    if(!rank) {
        LeeMatriz(u);
        for(int i = 1; i < P; i++)
            MPI_Send(u[N/P*i], N/P*N, MPI_DOUBLE, i, 0, MPI_COMM_WORLD);
    } else {
        MPI_Recv(u, N/P*N, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD, &status);
    }

    // calcular estadísticas
    for(int i=0; i<N/P; i++)
        for(int j=0; j<N; j++){
            media = media + u[i][j];
            if(u[i][j] > max) max = u[i][j];
            if(u[i][j] < min) min = u[i][j];
        }

    if(!rank) {
        for(int i = 1; i < P; i++) {
            MPI_Recv(buf, 3, MPI_DOUBLE, MPI_ANY_SOURCE, 0, MPI_COMM_WORLD, &status);
            media = media + buf[0];
            if(buf[1] > max) max = buf[1];
            if(buf[2] < min) min = buf[2];
        }
        media = media / (N*N);
        max_desv = ((max - media) > (media - min)) ? (max - media) : (media - min);
        for(int i = 1; i < P; i++)
            MPI_Send(&max_desv, 1, MPI_DOUBLE, i, 0, MPI_COMM_WORLD);
    } else {
        buf[0] = media;
        buf[1] = max;
        buf[2] = min;
        MPI_Send(buf, 3, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD);
        MPI_Recv(&max_desv, 1, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD, &status);
    }

    // modificar matriz
    for(int i=0; i<N/P; i++)
        for(int j=0; j<N; j++){
            u[i][j] = u[i][j] - max_desv;
        }

    if(!rank) {
        for(int i = 1; i < P; i++)
            MPI_Recv(u[N/P*i], N/P*N, MPI_DOUBLE, i, 0, MPI_COMM_WORLD, &status);
        EscribeMatriz(u);
    } else
        MPI_Send(u, N/P*N, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD);

    MPI_Finalize();
}

```

c)

```
void main(int argc, char *argv[]) {
    int rank, P;
    MPI_Status status;
    double max_desv, media = 0., max = 0., min = 0.;
    double u[N][N], buf;

    MPI_Init(&argc, &argv);
    MPI_Comm_size(&P, MPI_COMM_WORLD);
    MPI_Comm_rank(&rank, MPI_COMM_WORLD);

    if(!rank) {
        LeeMatriz(u);
    }
    MPI_Scatter(u, N/P*N, MPI_DOUBLE, rank ? u : MPI_IN_PLACE, N/P*N, MPI_DOUBLE,
        0, MPI_COMM_WORLD);

    // calcular estadísticas
    for(int i=0; i<N/P; i++)
        for(int j=0; j<N; j++){
            media = media + u[i][j];
            if(u[i][j] > max) max = u[i][j];
            if(u[i][j] < min) min = u[i][j];
        }

    MPI_Allreduce(&media, &buf, 1, MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);
    media = buf / (N*N);
    MPI_Allreduce(&max, &buf, 1, MPI_DOUBLE, MPI_MAX, MPI_COMM_WORLD);
    max = buf;
    MPI_Allreduce(&min, &buf, 1, MPI_DOUBLE, MPI_MIN, MPI_COMM_WORLD);
    min = buf;
    max_desv = ((max - media) > (media - min)) ? (max - media) : (media - min);

    // modificar matriz
    for(int i=0; i<N/P; i++)
        for(int j=0; j<N; j++){
            u[i][j] = u[i][j] - max_desv;
        }

    MPI_Gather(rank ? u : MPI_IN_PLACE, N/P*N, MPI_DOUBLE, u, N/P*N, MPI_DOUBLE,
        0, MPI_COMM_WORLD);
    if(!rank) {
        EscribeMatriz(u);
    }

    MPI_Finalize();
}
```